LOVELY PROFESSIONAL UNIVERSITY
NAAC Accredited 'A+' Grade | QS Ranked | NIRF Top 100

Department of Computer Science and Engineering
Phagwara, Punjab — 144411 | www.lpu.in

MAJOR PROJECT DOCUMENTATION

LuxeShop E-Commerce Platform

Deployment using Docker, Kubernetes, Jenkins CI/CD, Terraform, AWS ECR, Prometheus & Grafana

13 Chapters	7+ Technologies	Full-Stack App	End-to-End CI/CD	Cloud-Native Deploy

Name	Malla Bala Raju
Program	B.Tech — Computer Science Engineering
Academic Year	2025 – 2026
Institution	Lovely Professional University, Phagwara, Punjab
Faculty Guide	Dr. Chavi Ralhan (12086) Assistant Professor

Certificate

This is to certify that the project titled:

"LuxeShop E-Commerce Deployment using Docker, Kubernetes, Jenkins CI/CD, Terraform, AWS ECR, Prometheus & Grafana"

has been successfully completed by **Malla Bala Raju** in partial fulfillment of the requirements for the Degree of Bachelor of Technology in Computer Science and Engineering from Lovely Professional University, Phagwara, Punjab.

The project work embodies original research and independent work carried out by the student under the guidance of the Faculty Supervisor. The results reported herein are genuine and have not been submitted elsewhere for any award or degree.

Declaration by the Student

I, **Malla Bala Raju**, a student of Bachelor of Technology in Computer Science and Engineering at Lovely Professional University, Phagwara, Punjab, hereby solemnly declare that:

1. The project entitled "LuxeShop E-Commerce Deployment using Docker, Kubernetes, Jenkins CI/CD, Terraform, AWS ECR, Prometheus & Grafana" is my own original work and has been carried out under the supervision and guidance of my Faculty Guide.

2. The work presented in this documentation has not been submitted, in full or in part, to any other university or institution for the award of any other degree, diploma, or fellowship.

3. All external sources, references, documentation, research papers, and online tutorials consulted during the course of this project have been duly acknowledged.

4. I understand and agree that any misrepresentation of facts contained herein may result in disciplinary action, including cancellation of the degree.

5. The project implementation, architecture, deployment, troubleshooting, and all technical work documented in this report are authentic representations of work actually performed by me.

Signature: Malla Bala Raju

Name:	Malla Bala Raju
Program:	B.Tech — Computer Science Engineering
Date:	29-05-2026
Place:	Lovely Professional University, Phagwara, Punjab

Acknowledgement

I would like to express my sincere gratitude and heartfelt appreciation to all those who have supported, guided, and encouraged me throughout the course of this project.

I am deeply grateful to my **Faculty Guide and Supervisor**, whose constant mentorship, technical insights, and constructive feedback throughout the development and documentation of this project have been invaluable. The guidance provided helped me bridge the gap between academic learning and real-world industry DevOps practices.

I extend my sincere thanks to the **Head of the Department of Computer Science and Engineering** at Lovely Professional University, whose academic leadership and departmental facilities provided an excellent environment for conducting this project.

I acknowledge the invaluable contributions of the open-source community — particularly the developers and maintainers of **Docker, Kubernetes, Jenkins, Terraform, Prometheus, Grafana, Node.js, and MongoDB** — whose publicly available documentation significantly aided this project.

I also acknowledge **Amazon Web Services (AWS)** for providing robust cloud infrastructure services, including Amazon ECR, which formed a key component of the deployment pipeline for this project.

My sincere gratitude also goes to my **family and friends** for their continuous moral support and encouragement throughout this academic journey.

Malla Bala Raju

B.Tech — Computer Science Engineering

Lovely Professional University

Abstract

The rapid evolution of cloud-native technologies, containerization platforms, and DevOps methodologies has fundamentally transformed the manner in which modern software applications are designed, deployed, and maintained. This project presents a comprehensive, end-to-end implementation of a **production-style DevOps pipeline** for deploying the LuxeShop E-Commerce application using modern cloud and container technologies.

LuxeShop is a full-stack e-commerce platform consisting of a **static frontend** served via NGINX, a **RESTful backend API** built with Node.js and Express.js, and a **MongoDB** database for persistent data storage. All components are containerized using Docker and orchestrated on a local **Minikube Kubernetes** cluster, simulating a production-grade deployment environment.

The primary objective is to design and implement a complete DevOps workflow that automates every stage of the software delivery pipeline — from infrastructure provisioning and containerization to automated deployment, monitoring, and observability. Key technologies include **Terraform** for Infrastructure as Code, **Docker** and **Docker Compose** for containerization, **Kubernetes (Minikube)** for local orchestration, **Jenkins** for CI/CD automation, **AWS ECR** for Docker image management, and **Prometheus & Grafana** for real-time monitoring.

The project successfully demonstrates end-to-end automation encompassing infrastructure provisioning of AWS ECR repositories using Terraform, multi-container application management with Docker Compose, Kubernetes deployment with NodePort and ClusterIP service types, automated build-and-push pipelines in Jenkins triggered by GitHub webhooks through ngrok tunneling, and application health monitoring with Prometheus and Grafana dashboards.

The project concludes with a **fully operational, locally-hosted cloud-native application** on Minikube, complete with CI/CD automation, container orchestration, and a comprehensive monitoring and observability stack — providing practical, industry-aligned exposure to the complete DevOps lifecycle.

Keywords: DevOps · Docker · Kubernetes · Jenkins · Terraform · AWS ECR · Prometheus · Grafana · Minikube · CI/CD · MongoDB · Node.js · Express.js · NGINX · GitHub Webhooks · ngrok · Infrastructure as Code · Container Orchestration

Table of Contents

Certificate
Declaration by the Student
Acknowledgement
Abstract
Table of Contents
Chapter 1: Introduction
- 1.1 Project Overview
- 1.2 Problem Statement
- 1.3 Objectives
- 1.4 Scope
- 1.5 Existing Systems
- 1.6 Proposed System
- 1.7 Advantages

Chapter 2: Technology Stack
- 2.1 Overview
- 2.2 Complete Technology Stack
- 2.3 Technology Descriptions

Chapter 3: System Requirements
- 3.1 Hardware Requirements
- 3.2 Software Requirements

Chapter 4: System Architecture
- 4.1 Architecture Overview
- 4.2 Architecture Flow
- 4.3 Component Architecture
- 4.4 Kubernetes Networking

Chapter 5: Docker Documentation
Chapter 6: Kubernetes Deployment
Chapter 7: Jenkins CI/CD Pipeline
Chapter 8: Terraform Infrastructure
Chapter 9: Monitoring and Observability
Chapter 10: Common Errors and Troubleshooting
Chapter 11: Advantages of the System
Chapter 12: Future Enhancements
Chapter 13: Conclusion

Chapter 1: Introduction

1.1 Project Overview

LuxeShop is a modern, full-stack e-commerce platform deployed using a complete DevOps workflow. The project demonstrates real-world deployment automation practices using Docker, Kubernetes, Jenkins, Terraform, AWS ECR, Prometheus, and Grafana. It showcases how a multi-component web application — spanning frontend, backend, and database tiers — can be containerized, orchestrated, and continuously delivered through an automated pipeline with built-in monitoring.

The deployment targets a local **Minikube Kubernetes cluster**, making the project highly accessible for academic and development environments while faithfully replicating the architecture and workflows found in production cloud deployments. GitHub webhooks, tunneled through **ngrok**, enable real-time CI/CD triggers even from a local Jenkins server.

1.2 Problem Statement

Traditional application deployment approaches present complex, recurring challenges that significantly hinder development velocity and operational reliability:

- Manual deployment processes are slow, error-prone, and produce inconsistent results across environments.
- Lack of environment parity between development and production leads to the classic 'works on my machine' failures.
- Absence of container orchestration means no automatic scaling, self-healing, or rolling update capabilities.
- Without CI/CD automation, every release requires manual intervention, increasing the risk of human error.
- Poor or absent monitoring leaves teams blind to performance degradation and application health issues.
- Slow delivery cycles reduce competitive agility and delay user-facing improvements.

1.3 Objectives

Objective	Description
Automate Application Deployment	Replace manual deployment steps with a fully automated Jenkins CI/CD pipeline
Implement CI/CD using Jenkins	Configure Jenkins to trigger automated builds on every GitHub push event
Containerize with Docker	Package all application components (frontend, backend, database) as Docker images
Orchestrate with Kubernetes	Deploy and manage containers on a Minikube Kubernetes cluster
Manage Images via AWS ECR	Store and version Docker images in AWS Elastic Container Registry
Provision IaC with Terraform	Create AWS ECR repositories programmatically using Terraform HCL
Implement Monitoring	Collect real-time metrics with Prometheus and visualize them in Grafana

1.4 Scope of the Project

The project scope covers the full DevOps lifecycle for the LuxeShop application: local Kubernetes deployment using Minikube, Jenkins-based CI/CD automation with GitHub webhook integration via ngrok, Docker-based application packaging and multi-container management with Docker Compose, AWS ECR repository provisioning using Terraform, and a complete monitoring and observability stack using Prometheus and Grafana.

1.5 Existing Systems and Limitations

Approach	Description	Limitations
Manual Deployment	Direct server installation without automation	Error-prone, no rollback, slow and inconsistent
Simple Docker	Containers without orchestration or clustering	No scaling, no self-healing, manual lifecycle management
Heroku / PaaS	Platform-as-a-Service managed hosting	Vendor lock-in, limited customization, restricted control
No CI/CD	Manual build and deploy on every code change	High human error rate, slow releases, no audit trail
No Monitoring	Deployment without metrics or observability	Blind to issues, slow incident detection and response

1.6 Proposed System

- Infrastructure is provisioned automatically through **Terraform**, creating AWS ECR repositories in a repeatable, version-controlled manner.
- Applications are containerized using **Docker** and managed locally with **Docker Compose**, providing full environment consistency.
- **Kubernetes on Minikube** provides container orchestration, pod management, and service discovery for the local deployment.
- The **Jenkins CI/CD pipeline**, triggered by GitHub webhooks via ngrok, automates Docker image build, tag, and push operations on every commit.
- **Prometheus and Grafana** provide real-time application health monitoring, metric collection, and visualization.

1.7 Advantages of the Proposed System

Advantage	Description
Automated Deployment	End-to-end automation from code push to running containers in Kubernetes
Faster Release Cycles	Jenkins pipeline reduces deployment time from minutes to seconds
Improved Scalability	Kubernetes enables horizontal scaling of individual application components
Easy Monitoring	Prometheus and Grafana provide instant visibility into application health
High Availability	Kubernetes self-healing restarts failed pods automatically
Better Debugging	Container logs, Kubernetes events, and Prometheus metrics accelerate root cause analysis
Production Workflow	Mirrors industry-standard DevOps practices used at leading tech organizations
Easier Maintenance	Declarative Kubernetes manifests and Terraform IaC simplify updates and rollbacks

Chapter 2: Technology Stack

2.1 Overview

This project employs a carefully selected set of modern technologies spanning frontend hosting, backend API development, database management, containerization, container orchestration, infrastructure provisioning, CI/CD automation, and monitoring. Each technology was chosen for its industry adoption, community maturity, and suitability for demonstrating a complete, realistic DevOps workflow.

2.2 Complete Technology Stack

Category	Technology	Purpose
Frontend	HTML, CSS, JavaScript, NGINX	Static web app served via NGINX reverse proxy
Backend	Node.js, Express.js	RESTful API for e-commerce operations
Database	MongoDB	NoSQL document store for product and order data
Containerization	Docker 24.x	Application packaging into portable containers
Orchestration	Kubernetes	Container lifecycle management and service discovery
Kubernetes Platform	Minikube	Local single-node Kubernetes cluster for development
CI/CD	Jenkins 2.x LTS	Automated build, tag, and push pipeline
IaC	Terraform 1.x	Programmatic provisioning of AWS ECR repositories
Container Registry	AWS ECR	Secure Docker image storage and versioning
Monitoring	Prometheus 2.x	Time-series metrics collection and alerting
Visualization	Grafana 10.x	Interactive dashboards for metrics visualization
Version Control	GitHub	Source code repository and webhook trigger source
Webhook Tunnel	ngrok	Exposes local Jenkins to the public internet for webhooks

2.3 Technology Descriptions

2.3.1 Docker

Docker is an open-source containerization platform that packages applications and their dependencies into lightweight, isolated containers. In this project, Docker containerizes all five components: the NGINX frontend, the Node.js/Express backend, MongoDB, Prometheus, and Grafana. Docker Compose manages the multi-container setup for local development and testing, defining all services, networks, and volumes in a single declarative YAML file.

2.3.2 Kubernetes and Minikube

Kubernetes is an open-source container orchestration platform maintained by the CNCF that automates deployment, scaling, and management of containerized applications. Minikube creates a single-node Kubernetes cluster on a local machine, making it ideal for development and academic environments. In this project, Kubernetes manages the frontend, backend, and MongoDB pods using Deployment and Service manifests.

2.3.3 Jenkins

Jenkins is an open-source automation server that enables continuous integration and continuous delivery through programmable pipeline automation. GitHub webhooks (tunneled via ngrok) notify Jenkins of code pushes, automatically triggering the pipeline stages: repository checkout, Terraform init/apply, Docker image build, tagging, and pushing to AWS ECR.

2.3.4 Terraform

Terraform is an open-source Infrastructure as Code tool by HashiCorp that enables developers to define and provision cloud infrastructure using a declarative language (HCL). In this project, Terraform is scoped to provisioning the frontend and backend AWS ECR repositories, demonstrating how infrastructure can be version-controlled, reviewed, and applied consistently without manual AWS Console operations.

2.3.5 AWS Elastic Container Registry (ECR)

AWS ECR is a fully managed Docker container registry service that stores, versions, and distributes container images. ECR integrates natively with AWS IAM for secure image access control, and with Kubernetes for image pull authentication. Jenkins pushes tagged frontend and backend Docker images to their respective ECR repositories after each successful build.

2.3.6 Prometheus

Prometheus is an open-source monitoring and alerting system that collects time-series metrics by scraping configured HTTP endpoints at regular intervals. It provides PromQL — a powerful query language for aggregating and analyzing metrics. In this project, Prometheus is deployed as a Docker container and configured to scrape application metrics, HTTP request counts, container health, and backend performance indicators.

2.3.7 Grafana

Grafana is an open-source observability and visualization platform that connects to Prometheus as a data source to render rich, interactive dashboards. Grafana displays panels tracking HTTP request rates, pod health status, CPU utilization, and memory usage — giving operators instant visibility into application performance and infrastructure health.

2.3.8 ngrok

ngrok is a secure tunneling tool that creates a public HTTPS URL pointing to a locally running service — in this case, the Jenkins server. Since a local Jenkins instance is not directly reachable by GitHub's webhook system, ngrok bridges the gap by exposing the local Jenkins endpoint to the public internet, enabling real-time GitHub-to-Jenkins webhook delivery without requiring a cloud-hosted CI server.

Chapter 3: System Requirements

3.1 Hardware Requirements

Component	Minimum Specification	Notes
Processor	Intel Core i5 or above	Recommended: Intel Core i7 / AMD Ryzen 7
RAM	Minimum 8 GB	Recommended: 16 GB (Minikube is memory-intensive)
Storage	Minimum 50 GB SSD	Recommended: 100 GB SSD
Network	Stable Internet Connection	Required for AWS ECR push and ngrok tunneling
Operating System	Windows 11 with WSL2	Ubuntu 22.04 LTS also supported
Display	1280 x 720 minimum	Recommended: 1920 x 1080 for Grafana dashboards

NOTE Minikube allocates a significant portion of host RAM for the virtual Kubernetes cluster. 8 GB is the minimum; 16 GB is strongly recommended for running all project containers simultaneously without performance degradation.

3.2 Software Requirements

Software	Purpose / Description
Windows 11 / WSL2	Host operating system with Linux subsystem support
Docker Desktop	Container runtime and Docker Compose for local development
Minikube	Local single-node Kubernetes cluster
kubectl	Kubernetes command-line interface tool
Terraform 1.x	Infrastructure as Code provisioning for AWS ECR
Jenkins 2.x LTS	CI/CD automation server
AWS CLI 2.x	AWS command-line interface for ECR authentication
Git 2.x	Source control and GitHub integration
ngrok	Webhook tunnel to expose local Jenkins to GitHub
Node.js 18+	Backend application runtime and Jenkins build dependency
Java JDK 17+	Jenkins server runtime environment

Chapter 4: System Architecture

4.1 Architecture Overview

The LuxeShop system architecture follows a cloud-native, microservices-oriented design pattern organized across four layers: **Developer Workstation**, **CI/CD Automation**, **Container Registry**, and **Local Kubernetes Cluster with Monitoring**. Each layer communicates through well-defined interfaces, ensuring separation of concerns and enabling independent development of each component.

4.2 Architecture Flow

Step 1 — Code Push	Developer writes code and pushes changes to the GitHub repository.
Step 2 — Webhook Trigger	GitHub sends a POST webhook to the Jenkins server via the ngrok public URL.
Step 3 — Pipeline Start	Jenkins receives the webhook and automatically initiates the CI/CD pipeline.
Step 4 — Terraform Init	Jenkins runs terraform init and terraform apply to provision ECR repositories if not yet created.
Step 5 — Docker Build	Jenkins executes docker-compose build to create fresh Docker images for all services.
Step 6 — ECR Auth	Jenkins authenticates the Docker daemon with AWS ECR using aws ecr get-login-password.
Step 7 — Image Tag	Jenkins tags the built images with the full AWS ECR registry URI.
Step 8 — ECR Push	Jenkins pushes the tagged frontend and backend images to their AWS ECR repositories.
Step 9 — K8s Deploy	Kubernetes on Minikube pulls the updated images and creates / updates pods.
Step 10 — App Running	LuxeShop frontend and backend are accessible via Minikube NodePort services.
Step 11 — Metrics Scrape	Prometheus scrapes configured endpoints at regular intervals.
Step 12 — Visualization	Grafana reads Prometheus metrics and updates monitoring dashboards in real time.

4.3 Component Architecture

4.3.1 Frontend Component

The frontend is a static HTML/CSS/JavaScript web application served by an NGINX container. NGINX serves static files for direct page requests and acts as a reverse proxy, forwarding all API calls (prefixed with /api/) to the backend service. In Kubernetes, the frontend is deployed as a Deployment and exposed via a NodePort Service, making it accessible from the host machine using the Minikube IP address.

4.3.2 Backend Component

The backend is a RESTful API built with Node.js and Express.js, handling all business logic for the LuxeShop e-commerce platform — including product catalogue management, user operations, and order processing. It connects to MongoDB using Mongoose ODM. The backend is deployed as a Kubernetes Deployment and exposed within the cluster via a ClusterIP Service, accessible only to the NGINX frontend proxy.

4.3.3 MongoDB Database Component

MongoDB serves as the primary NoSQL data store for all LuxeShop application data. It is deployed as a Kubernetes Deployment with a PersistentVolumeClaim to ensure data durability across pod restarts. MongoDB is exposed internally via a ClusterIP Service, accessible only by the Node.js backend within the cluster network.

4.3.4 Monitoring Stack

Prometheus and Grafana are deployed as Docker containers alongside the Kubernetes cluster. Prometheus scrapes metrics from the application and containers at configurable intervals. Grafana connects to Prometheus as a data source and displays pre-configured dashboards tracking request rates, pod health, CPU usage, and memory consumption.

4.4 Kubernetes Networking

Kubernetes networking within the Minikube cluster assigns each pod a unique internal IP address. Services provide stable virtual IPs and DNS names that abstract individual pod addresses, enabling reliable inter-service communication even as pods are rescheduled. The NGINX frontend communicates with the Node.js backend using the Kubernetes DNS name backend-service. External browser access is provided via NodePort services through the Minikube cluster IP.

> **NOTE** Use the command 'minikube service frontend-service --url' to obtain the externally accessible URL for the LuxeShop frontend. Minikube assigns a cluster IP dynamically on each start.

Chapter 5: Docker Documentation

5.1 Docker Architecture

The LuxeShop application uses Docker to containerize each service independently. Docker Compose orchestrates the multi-container setup for local development, defining all services, their configurations, network connections, and volume mounts in a single docker-compose.yml file. Each service runs in its own isolated container, communicating through a shared Docker bridge network.

5.2 Service Overview

Service	Base Image	Port	Purpose
frontend	nginx:alpine	80	Serves static files and proxies API calls to backend
backend	node:18-alpine	5000	Node.js/Express RESTful API
mongo	mongo:6	27017	MongoDB NoSQL database
prometheus	prom/prometheus:latest	9090	Metrics collection and storage
grafana	grafana/grafana:latest	3001	Metrics visualization dashboards

5.3 Docker Compose Configuration

```
# docker-compose.yml
version: '3.8'
services:
  frontend:
    build: ./frontend
    ports:
      - '80:80'
    depends_on:
      - backend

  backend:
    build: ./backend
    ports:
      - '5000:5000'
    environment:
      - MONGO_URI=mongodb://mongo:27017/luxeshop
    depends_on:
      - mongo

  mongo:
    image: mongo:6
    volumes:
      - mongo_data:/data/db

  prometheus:
    image: prom/prometheus:latest
    ports:
      - '9090:9090'
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml

  grafana:
    image: grafana/grafana:latest
    ports:
      - '3001:3000'
    depends_on:
      - prometheus
```

```
volumes:
  mongo_data:
```

5.4 Key Docker Commands

Command	Description
docker-compose build	Build all service images defined in docker-compose.yml
docker-compose up -d	Start all containers in detached background mode
docker-compose down	Stop and remove all running containers
docker ps	List all currently running containers
docker logs <container>	View stdout/stderr logs for a specific container
docker exec -it <c> sh	Open an interactive shell inside a running container
docker images	List all locally available Docker images

Chapter 6: Kubernetes Deployment

6.1 Kubernetes Architecture

The LuxeShop application is deployed on a local Minikube Kubernetes cluster. Kubernetes manages the desired state of the application through Deployment manifests, ensuring that the specified number of replicas for each service are always running. Services provide stable network identities and load balancing for pods, abstracting away individual pod IP addresses.

6.2 Kubernetes Services

Service Name	Type	Port	Purpose
frontend-service	NodePort	80	Exposes the NGINX frontend to external browser traffic via Minikube IP
backend-service	ClusterIP	5000	Internal-only access to the Node.js backend for the NGINX proxy
mongo-service	ClusterIP	27017	Internal-only access to MongoDB for the Node.js backend

6.3 Deployment Manifest

Each application component is defined as a Kubernetes Deployment, ensuring declarative management of pod replicas, container images, resource limits, and restart policies:

```
# backend-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
      - name: backend
        image: <ecr-registry>/backend:latest
        ports:
        - containerPort: 5000
        env:
        - name: MONGO_URI
          value: mongodb://mongo-service:27017/luxeshop
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  selector:
    app: backend
  ports:
  - port: 5000
    targetPort: 5000
  type: ClusterIP
```

6.4 Key kubectl Commands

Command	Description
kubectl apply -f k8s/	Apply all manifests in the k8s directory
kubectl get pods	List all running pods and their statuses
kubectl get services	List all services and their cluster/node IPs
kubectl describe pod <name>	View detailed pod information and events
kubectl logs <pod-name>	Stream container logs for debugging
kubectl rollout restart deployment/<n>	Force pod recreation to pick up new image
minikube service frontend-service --url	Get the external URL for the frontend NodePort
kubectl get events --sort-by=.lastTimestamp	View cluster events chronologically

Chapter 7: Jenkins CI/CD Pipeline

7.1 Overview

The Jenkins CI/CD pipeline is the automation backbone of the LuxeShop deployment workflow. It eliminates manual build and push steps by automatically executing a defined sequence of stages every time a developer pushes code to GitHub. The pipeline integrates Terraform for infrastructure management, Docker for image building, and AWS ECR for image distribution — all triggered automatically through GitHub webhooks tunneled via ngrok.

7.2 Pipeline Stages

Stage	Description	Key Command
1. Clone Repository	Jenkins checks out the latest code from the GitHub repository	git clone / git pull
2. Terraform Init	Initialises Terraform working directory and downloads AWS provider	terraform init
3. Terraform Apply	Provisions AWS ECR repositories for frontend and backend images	terraform apply -auto-approve
4. Docker Build	Builds fresh Docker images for frontend and backend services	docker-compose build
5. Docker Tag	Tags built images with the full AWS ECR registry URI	docker tag
6. Push to ECR	Authenticates and pushes tagged images to AWS ECR repositories	docker push

7.3 Jenkinsfile — Complete Pipeline

```
pipeline {
  agent any
  environment {
    AWS_REGION    = 'ap-south-1'
    ECR_REGISTRY  = '<account-id>.dkr.ecr.ap-south-1.amazonaws.com'
    FRONTEND_IMG  = "${ECR_REGISTRY}/luxeshop-frontend"
    BACKEND_IMG   = "${ECR_REGISTRY}/luxeshop-backend"
  }
  stages {
    stage('Clone Repository') {
      steps { git branch: 'main', url: 'https://github.com/mallabalaraju/luxeshop' }
    }
    stage('Terraform Init') {
      steps { dir('terraform') { sh 'terraform init' } }
    }
    stage('Terraform Apply') {
      steps { dir('terraform') { sh 'terraform apply -auto-approve' } }
    }
    stage('Docker Build') {
      steps { sh 'docker-compose build --no-cache' }
    }
    stage('Docker Tag') {
      steps {
        sh 'docker tag luxeshop-frontend:latest ${FRONTEND_IMG}:latest'
        sh 'docker tag luxeshop-backend:latest  ${BACKEND_IMG}:latest'
      }
    }
    stage('Push to ECR') {
      steps {
```

```
        sh 'aws ecr get-login-password --region ${AWS_REGION} | docker login --username AWS --
password-stdin ${ECR_REGISTRY}'
        sh 'docker push ${FRONTEND_IMG}:latest'
        sh 'docker push ${BACKEND_IMG}:latest'
      }
    }
  }
  post {
    success { echo 'Pipeline succeeded -- images pushed to ECR!' }
    failure { echo 'Pipeline failed -- check stage logs.' }
  }
}
```

7.4 GitHub Webhook with ngrok

Because Jenkins runs on a local machine, it is not directly reachable by GitHub's webhook service. ngrok solves this by creating a secure public tunnel to the local Jenkins port. The workflow is:

- Start Jenkins locally on port 8080.
- Run **ngrok http 8080** to generate a public forwarding URL (e.g., https://abc123.ngrok.io).
- In GitHub: go to **Settings > Webhooks > Add webhook**.
- Set Payload URL to: **https://abc123.ngrok.io/github-webhook/**
- Set Content type to **application/json** and trigger on **Push events**.
- Click **Add webhook**. Every subsequent git push will automatically trigger the Jenkins pipeline.

WARNING ngrok generates a new public URL on every restart. Update the GitHub webhook Payload URL each time ngrok is restarted. For persistent deployments, use a paid ngrok plan with a static domain or host Jenkins on a cloud server.

Chapter 8: Terraform Infrastructure

8.1 Overview

Terraform is used in this project to provision the AWS infrastructure required for Docker image storage. Specifically, Terraform creates and manages the two AWS ECR repositories — one for the frontend image and one for the backend image — using HashiCorp Configuration Language (HCL). This approach ensures that infrastructure creation is reproducible, version-controlled, and auditable.

8.2 Terraform Resources

Terraform Resource	Purpose	Repository Name
aws_ecr_repository.frontend	ECR repository for NGINX frontend Docker images	luxeshop-frontend
aws_ecr_repository.backend	ECR repository for Node.js backend Docker images	luxeshop-backend

8.3 Terraform Configuration

```
# terraform/main.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_ecr_repository" "frontend" {
  name                 = "luxeshop-frontend"
  image_tag_mutability = "MUTABLE"
  image_scanning_configuration { scan_on_push = true }
}

resource "aws_ecr_repository" "backend" {
  name                 = "luxeshop-backend"
  image_tag_mutability = "MUTABLE"
  image_scanning_configuration { scan_on_push = true }
}

output "frontend_repo_url" {
  value = aws_ecr_repository.frontend.repository_url
}

output "backend_repo_url" {
  value = aws_ecr_repository.backend.repository_url
}
```

Chapter 9: Monitoring and Observability

9.1 Monitoring Architecture

The monitoring stack implements a **pull-based metrics collection model**. Prometheus actively scrapes configured HTTP metric endpoints at regular intervals and stores time-series data in its internal database. Grafana connects to Prometheus as a data source, executes PromQL queries, and renders the results as real-time interactive dashboards. Both tools run as Docker containers alongside the application.

9.2 Prometheus Configuration

```
# prometheus.yml
global:
  scrape_interval:     15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'luxeshop-backend'
    static_configs:
      - targets: ['backend:5000']
    metrics_path: /metrics

  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']
```

9.3 Prometheus Metrics Collected

Metric Name	Description
http_requests_total	Counter — total HTTP requests by method, route, and status code
http_request_duration_seconds	Histogram — request processing duration in seconds
process_cpu_seconds_total	Counter — total CPU time consumed by the backend process
process_resident_memory_bytes	Gauge — current resident memory usage of the backend
up	Binary gauge — 1 if the scrape target is healthy, 0 if unreachable

9.4 Key PromQL Queries

PromQL Query	Description
up	Target health status (1 = up, 0 = down)
rate(http_requests_total[1m])	Per-second request rate over last 1 minute
rate(http_requests_total{status=~"5.."}[1m])	Per-second server error rate
rate(process_cpu_seconds_total[5m])	CPU usage rate over last 5 minutes
process_resident_memory_bytes / 1024 / 1024	Memory usage in megabytes

9.5 Grafana Dashboard Panels

Panel Title	Panel Type	PromQL Metric
Backend Status	Stat Panel	up
HTTP Request Rate	Time Series	rate(http_requests_total[1m])
Error Rate (5xx)	Gauge	rate(http_requests_total{status=~"5.."}[1m])
CPU Usage	Time Series	rate(process_cpu_seconds_total[5m])
Memory Usage (MB)	Stat Panel	process_resident_memory_bytes / 1024 / 1024
Pod Health	Table	kube_pod_status_ready

Chapter 10: Common Errors and Troubleshooting

10.1 Overview

This chapter documents the most common errors encountered during development and deployment of the LuxeShop project, along with their root causes, confirmed solutions, and prevention strategies.

10.2 Docker Errors

ERROR **ERROR DOC-01: Port Already Allocated** — Bind for 0.0.0.0:80 failed — address already in use. Another process is bound to that port. Root Cause: Another service (IIS, Apache, or a previous Docker container) is already using port 80. Fix: Find the conflicting process with 'sudo lsof -i :80' (Linux) or 'netstat -ano | findstr :80' (Windows), then stop it or change the port mapping in docker-compose.yml. Prevention: Document used port numbers and check for conflicts before starting the stack.

ERROR **ERROR DOC-02: Image Build Failure — Module Not Found** — npm ERR! Cannot find module 'express' during Docker build. Root Cause: The Dockerfile COPY instruction did not include node_modules, and npm install was not run during the build. Fix: Add 'RUN npm install' after 'COPY package*.json ./' in the Dockerfile. Prevention: Always use a multi-stage Dockerfile and ensure the build stage runs npm install.

10.3 Kubernetes Errors

ERROR **ERROR K8S-01: CrashLoopBackOff** — Pod repeatedly crashes immediately after starting. kubectl describe pod shows 'Back-off restarting failed container'. Root Cause: Application inside the container is failing to start — check container logs for the actual error. Common causes include missing environment variables or wrong image. Fix: Run 'kubectl logs <pod-name>' to see the container output. Correct the configuration issue and redeploy. Prevention: Test Docker images locally with docker run before deploying to Kubernetes.

ERROR **ERROR K8S-02: ImagePullBackOff** — Pod cannot start because Kubernetes cannot pull the specified Docker image from AWS ECR. Root Cause: AWS credentials are not configured in Jenkins, or the IAM user/role lacks ECR permissions. Fix: Add AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY as Jenkins Credentials. Attach AmazonEC2ContainerRegistryFullAccess to the IAM user. Prevention: Test AWS CLI authentication manually before configuring Jenkins: aws ecr describe-repositories

10.4 CI/CD Errors

ERROR **ERROR CI-01: ECR Authentication Failure** — Jenkins stage 'Push to ECR' fails with: An error occurred (RequestExpired) — Request has expired. Root Cause: AWS credentials are not configured in Jenkins, or the IAM user/role lacks ECR permissions. Fix: Add AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY as Jenkins Credentials (Secret text type). Attach AmazonEC2ContainerRegistryFullAccess to the IAM user. Prevention: Test AWS CLI authentication manually before configuring Jenkins.

ERROR **ERROR CI-02: ngrok Tunnel Expired — Webhook Not Triggering** — Jenkins pipeline is not triggered on GitHub push. GitHub webhook delivery shows 'Failed to connect to host'. Root Cause: ngrok session expired or was restarted, generating a new public URL. The old URL in GitHub webhook settings is no longer valid. Fix: Restart ngrok and copy the new forwarding URL. Update the

GitHub webhook Payload URL with the new ngrok address. Prevention: Use a paid ngrok plan with a static domain, or consider hosting Jenkins on an EC2 instance with a persistent public IP.

10.5 General Debugging Reference

Problem	Diagnostic Command
Pod not starting	kubectl describe pod <name> && kubectl logs <name>
Service unreachable	kubectl get svc && kubectl get endpoints
Image pull failure	kubectl describe pod <name> — check Events section
Check container env vars	kubectl exec -it <pod> -- env
All cluster events	kubectl get events --sort-by=.lastTimestamp
Prometheus target down	Open http://localhost:9090/targets in browser
ngrok status	Open http://localhost:4040 for ngrok tunnel inspector
Docker network issues	docker network inspect bridge

Chapter 11: Advantages of the System

11.1 System Advantages

Automated Deployment	Every git push triggers the Jenkins pipeline automatically, producing a consistent, reproducible deployment without any manual intervention.
Faster Release Cycles	The CI/CD pipeline compresses the build-test-deploy cycle from hours to minutes, enabling rapid iteration and feedback.
Improved Scalability	Kubernetes Deployments allow horizontal scaling of individual services by adjusting replica counts, enabling the application to handle variable traffic loads.
Easy Monitoring	Prometheus and Grafana provide real-time dashboards and alerting, giving operators instant visibility into application health without manual log parsing.
High Availability	Kubernetes automatically detects and restarts failed containers, ensuring the application remains available even when individual pods crash.
Better Debugging	Centralized metrics, container logs, and Kubernetes events significantly accelerate root cause analysis and incident resolution.
Production-Style Workflow	The architecture mirrors industry practices used at leading technology organizations, providing directly applicable, market-relevant experience.
Easier Maintenance	Declarative Kubernetes manifests and Terraform IaC enable straightforward updates, rollbacks, and environment reproductions.

Chapter 12: Future Enhancements

12.1 Planned Improvements

Enhancement	Description
Deploy on AWS EKS	Migrate from local Minikube to AWS Elastic Kubernetes Service for a fully managed, production-grade cloud deployment
Add Helm Charts	Package all Kubernetes manifests as Helm charts for parameterized, versioned deployments with environment-specific value overrides
Implement ArgoCD GitOps	Adopt a GitOps delivery model where Kubernetes state is continuously synchronized from the Git repository using ArgoCD
Horizontal Pod Autoscaling	Configure Kubernetes HPA to automatically scale pod replicas based on CPU or custom Prometheus metrics
HTTPS Ingress Controller	Deploy an NGINX Ingress Controller with cert-manager for TLS termination, replacing NodePort services
Centralized Logging (ELK)	Integrate Elasticsearch, Logstash, and Kibana for centralized log aggregation, search, and visualization
Database High Availability	Replace the single MongoDB pod with a replica set or migrate to AWS DocumentDB for production-grade database resilience
Security Scanning	Integrate Trivy or AWS ECR image scanning into the Jenkins pipeline to detect vulnerabilities before deployment
Blue-Green Deployments	Implement blue-green or canary deployment strategies to achieve zero-downtime releases with instant rollback capability
Static Webhook Domain	Replace ngrok with a cloud-hosted Jenkins instance on EC2 with an Elastic IP to eliminate the ngrok URL refresh requirement

Chapter 13: Conclusion

The LuxeShop DevOps project has successfully demonstrated the design and implementation of a complete, production-style cloud-native deployment workflow using Docker, Kubernetes, Jenkins, Terraform, AWS ECR, Prometheus, and Grafana. Every stated objective of the project was achieved, providing comprehensive hands-on exposure to the full spectrum of modern DevOps technologies and practices.

The implementation journey encompassed **containerizing a three-tier e-commerce application** using Docker and Docker Compose, **orchestrating the containers** on a local Minikube Kubernetes cluster with Deployment and Service manifests, **provisioning AWS infrastructure** through Terraform Infrastructure as Code, and **automating the complete build-and-push pipeline** through a Jenkins CI/CD server integrated with GitHub webhooks via ngrok tunneling.

The monitoring and observability stack — built with Prometheus for metrics collection and Grafana for visualization — provides real-time visibility into application health, HTTP request rates, CPU utilization, and memory consumption, simulating the operational dashboards used in production environments.

Common real-world challenges were encountered and resolved throughout the project lifecycle. The **ngrok URL expiry issue** highlighted the operational complexity of exposing local services to external systems and underscored the value of persistent, cloud-hosted CI infrastructure. The **Docker port allocation errors** reinforced the importance of careful resource planning and port management documentation. The **Kubernetes CrashLoopBackOff errors** provided practical experience in container log analysis and Kubernetes event debugging.

The final outcome is a **fully functional, locally-deployed cloud-native application** with automated CI/CD, container orchestration, infrastructure-as-code provisioning, and comprehensive monitoring — all implemented using the same tools and patterns employed by professional DevOps engineers at leading technology organizations worldwide.

This project has significantly strengthened practical competencies in containerization, orchestration, pipeline automation, infrastructure provisioning, and observability engineering. These skills are directly applicable to modern software engineering roles and form a strong foundation for advancing into cloud-native, platform engineering, and SRE career tracks.

End of Major Project Documentation

Malla Bala Raju | B.Tech Computer Science Engineering | Lovely Professional University | 2025-2026